University of Messina

Bachelor of Data Analysis

ACADEMIC YEAR - 2024/2025

Machine Learning

(Project report)

Supervisor:
Prof. Giacomo Fiumara

Student:
Samidhaben Parmar(540946)
Mohammad Owais(541462)
Syed fiaz ul hassan(541569)

Table of Content

Overview

In this project, we focused on building models that can predict. We cleaned the data, explored patterns, created useful features, trained different models, tuned them for better results, and selected the best ones based on their performance.

1. Dataset Exploration

The dataset consists of 9000 entries with 11 columns including 8 numerical features, 2 categorical features and a binary target variable.

Numerical Feature: Feature_1, Feature_2, Feature_3, Feature_4, Feature_5, Feature_6, Feature_7, Feature_8.

Categorical Feature: category_1 (contains values like “Above Average”, “Below Average” and “High”) and category_2 (contains values like “Region A”, “Region B”, “Region C”).

Target: Target variable.

1.1) Feature Descrip-on:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9000 entries, 0 to 8999
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   feature_1   9000 non-null   float64
 1   feature_2   9000 non-null   float64
 2   feature_3   8600 non-null   float64
 3   feature_4   9000 non-null   float64
 4   feature_5   9000 non-null   float64
 5   feature_6   8500 non-null   float64
 6   feature_7   9000 non-null   float64
 7   feature_8   9000 non-null   float64
 8   category_1  9000 non-null   object
 9   category_2  9000 non-null   object
 10  target      9000 non-null   int64
dtypes: float64(8), int64(1), object(2)
memory usage: 773.6+ KB
```

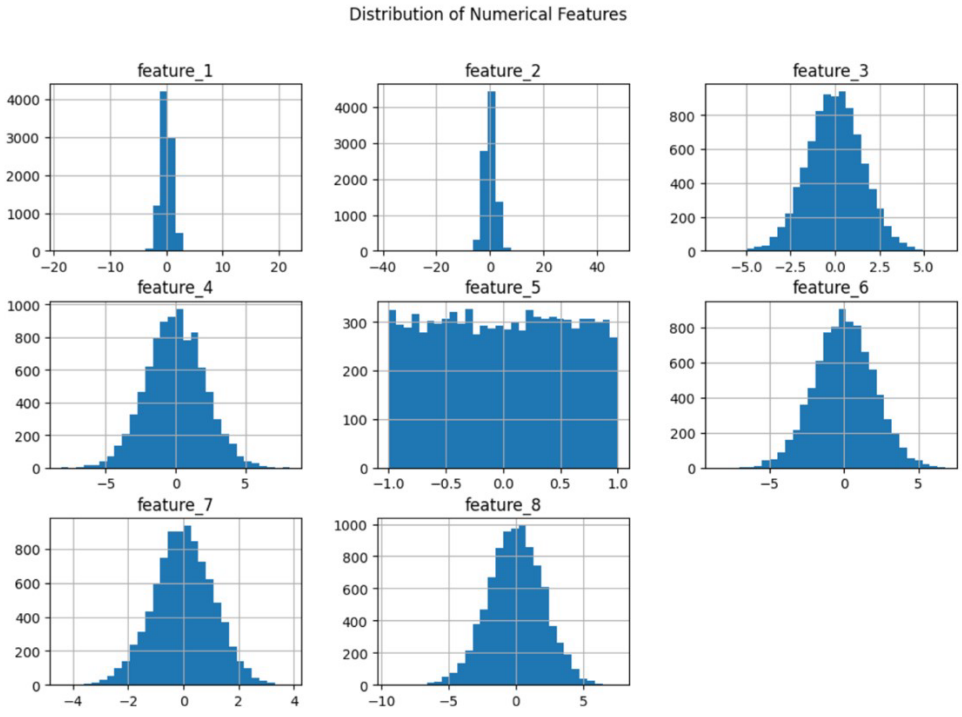
```
Value counts for category_1:

category_1
Low              2802
High             2763
Above Average    1727
Below Average    1708
Name: count, dtype: int64


Value counts for category_2:

category_2
Region B    3618
Region A    3551
Region C    1831
Name: count, dtype: int64
```

1.2) Missing Values:

'Missing values present in Feature_3 and Feature_6.

- “Feature_3” has 400 missing values.
- “Feature_6” has 500 missing values.

1.3) Feature Distribution:



- Most features like feature_3, feature_4, feature_6, feature_7, and feature_8 looks normal distributions.
- feature_5 shows a flat (uniform) distribution.
- feature_1 and feature_2 are tightly centered around zero with very small spread.
- Overall, most features are balanced without strong skewness, and scaling them would still be useful before training models.

2. Data Preprocessing

2.1) Handling the missing values

We utilize the functions “isnull()” to inspect null values. Then the null values were imputed using median of each column.

```
print(data.isnull().sum())

imputer = SimpleImputer(missing_values=np.nan, strategy='median')
data['feature_3'] = imputer.fit_transform(data['feature_3'].values.reshape(-1, 1))
data['feature_6'] = imputer.fit_transform(data['feature_6'].values.reshape(-1, 1))
```

Result: Missing count was reduced to zero.

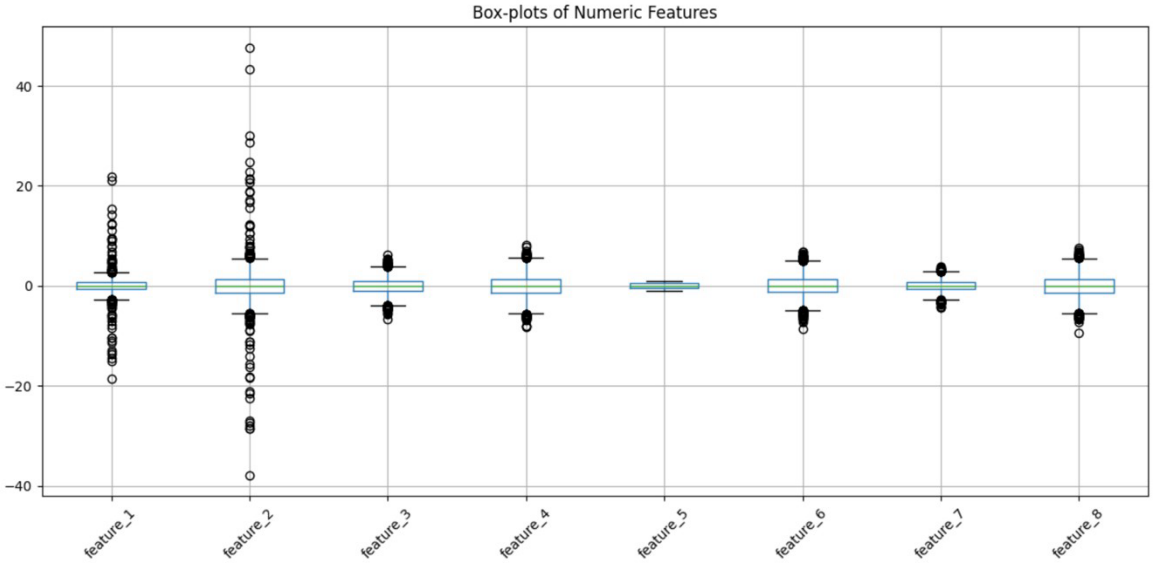
2.2) Label Encoding for the categorical features

To handle categorical features, we used Label Encoding. It converted the categories into numbers (0, 1, 2 ..), making them easier for machine learning models to understand. We applied this method to category_1 and category_2.

```
cat_features = ['category_1', 'category_2']

# Label encoding for categorical feature
label_encoder = LabelEncoder()
for feature in cat_features:
    data[feature] = label_encoder.fit_transform(data[feature])
```

2.3) Handling the Outliers

Outliers are datapoints that deviate significantly from the rest of the dataset and can have a substantial impact on analysis.

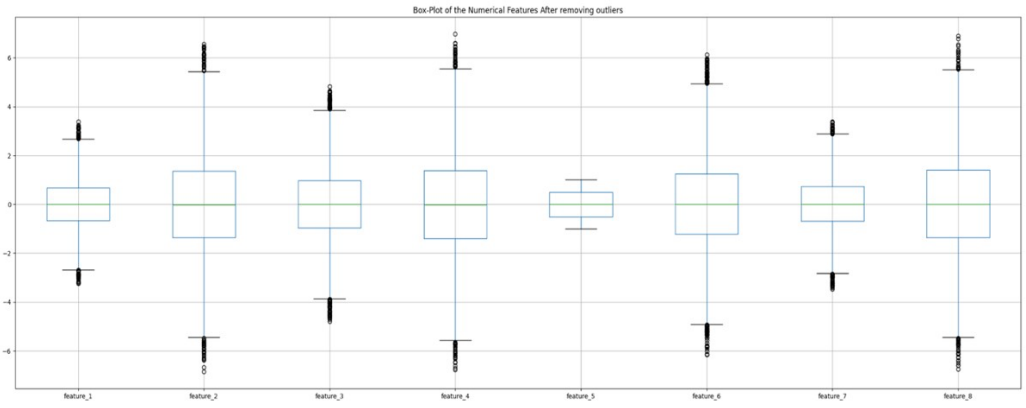


We found outliers in the data using the Interquartile Range (IQR) method. Values outside the range were considered outliers. feature_1, feature_2, feature_4, and feature_8 had significant outliers, while others had very few or none. These outliers were replaced with the median of the feature to reduce their effect.

Used IQR method (multiplier=2) to detect extreme observations.

```python
def iqr_outliers(dataset,feature_name, multiplier=2):

    Q1 = dataset[feature_name].quantile(0.25)
    Q3 = dataset[feature_name].quantile(0.75)

    IQR = Q3 - Q1

    lwr_bound = Q1 - multiplier * IQR
    upp_bound = Q3 + multiplier * IQR

    ls = dataset.index[np.logical_or(dataset[feature_name]<lwr_bound,
                                     dataset[feature_name]>upp_bound)]
    return ls #return the indexes

outliers_detected={}
for i in num_cols:
    outliers = iqr_outliers(data,i)
    outliers_detected[i] = outliers

    print('Variable',i)
    print(outliers)
    print(data[i].iloc[outliers])
    print('\n')
```

Outliers replaced with median values for each numeric feature.



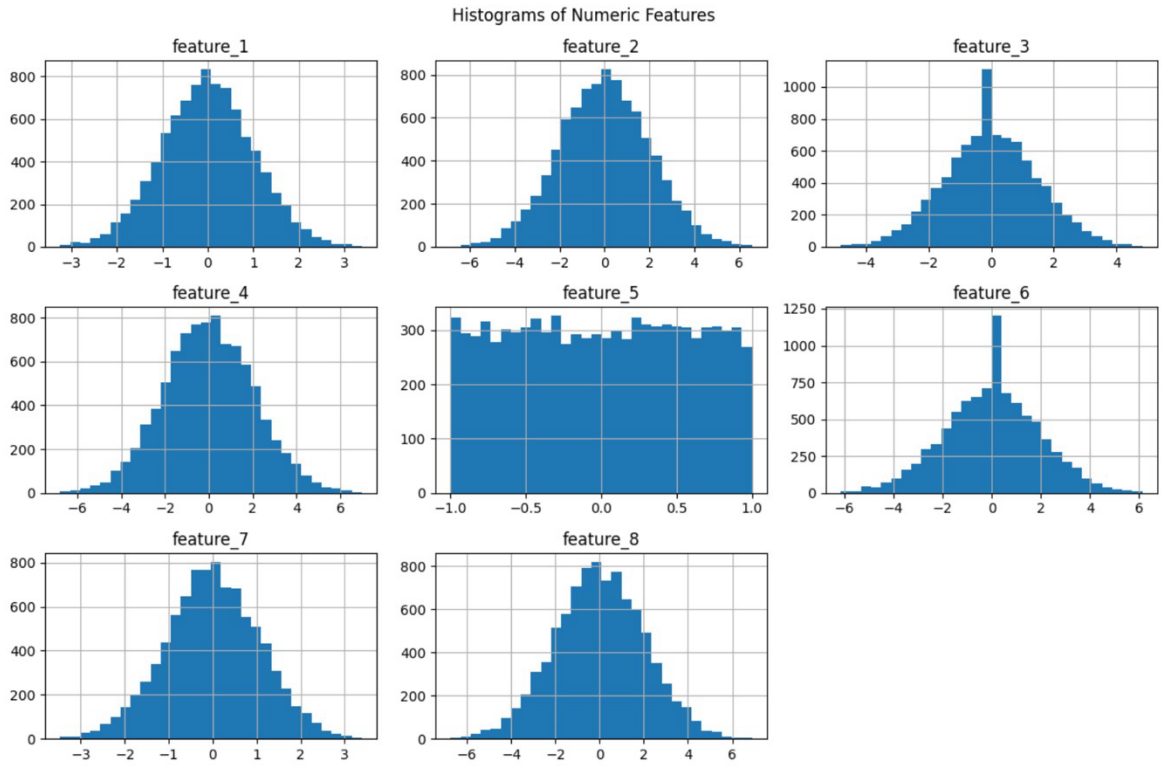
after that everything has tightened to roughly ±7, with only small “circles” beyond the whiskers.

Now, we got the cleaned dataset free from missing values, inconsistencies and treated outliers and with encoded categorical features.

3. EDA (Exploratory Data Analysis)

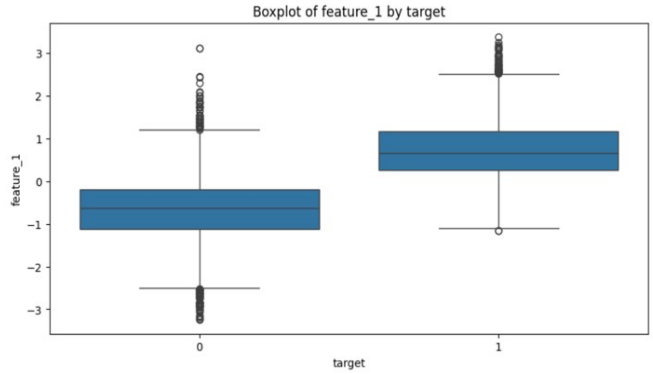
Exploratory Data Analysis (EDA) helps us take a closer look at the data before building models. It shows patterns, trends, and possible problems in the dataset. In this part, we used different graphs and tests to better understand the features and their relationship with the target.

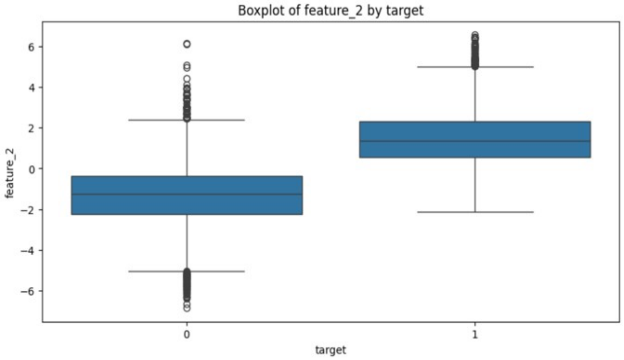
3.1) Distribution of Numerical Features (Histogram for and numerical features)

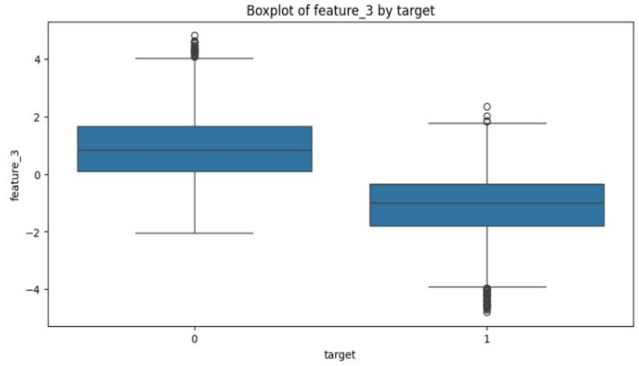


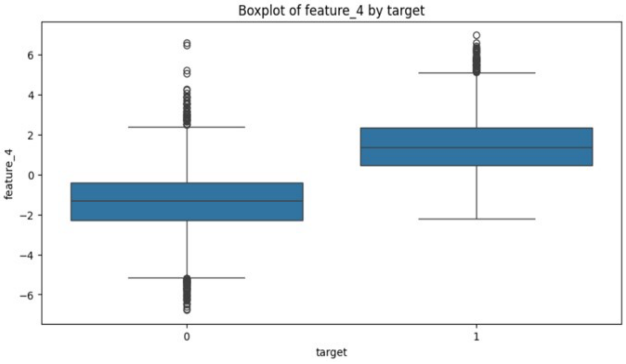
The numeric features mostly follow a normal distribution, except feature_5, which shows a uniform distribution, and feature_3 and feature_6, which have slight spikes around zero.

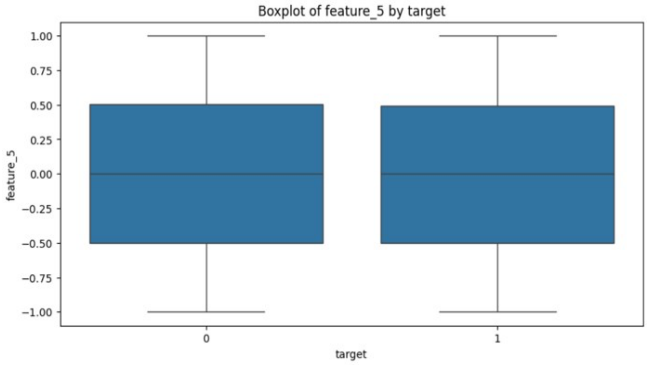
From the box-plots we will see the distribution of numerical feature with respect to the target. Features like feature_1, feature_2, and feature_3,feature_4 exhibited significant differences between the target classes, suggesting that these features are informative in predicting the target variable.

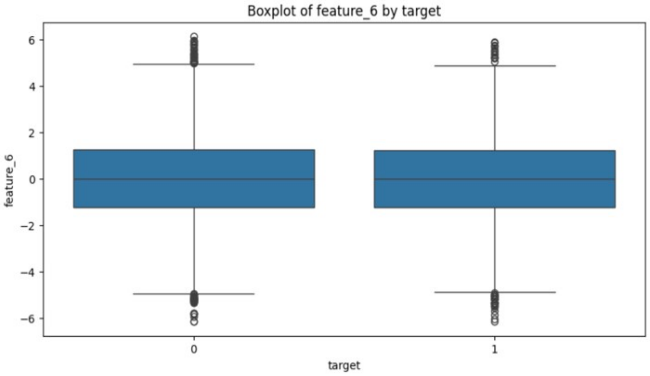


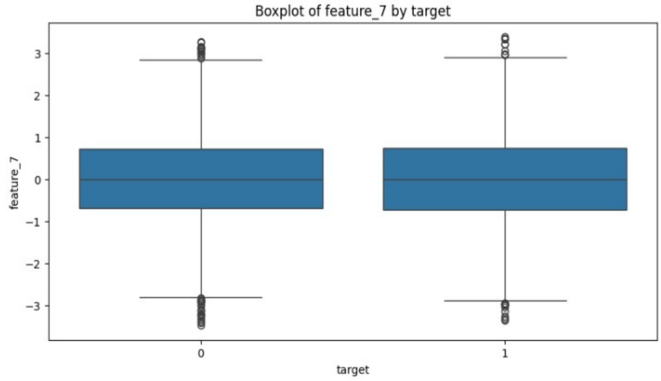


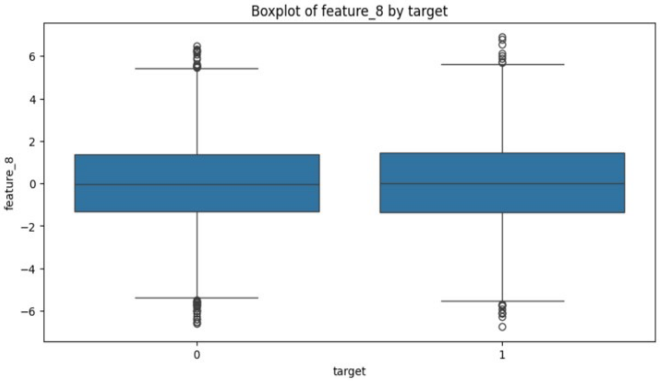






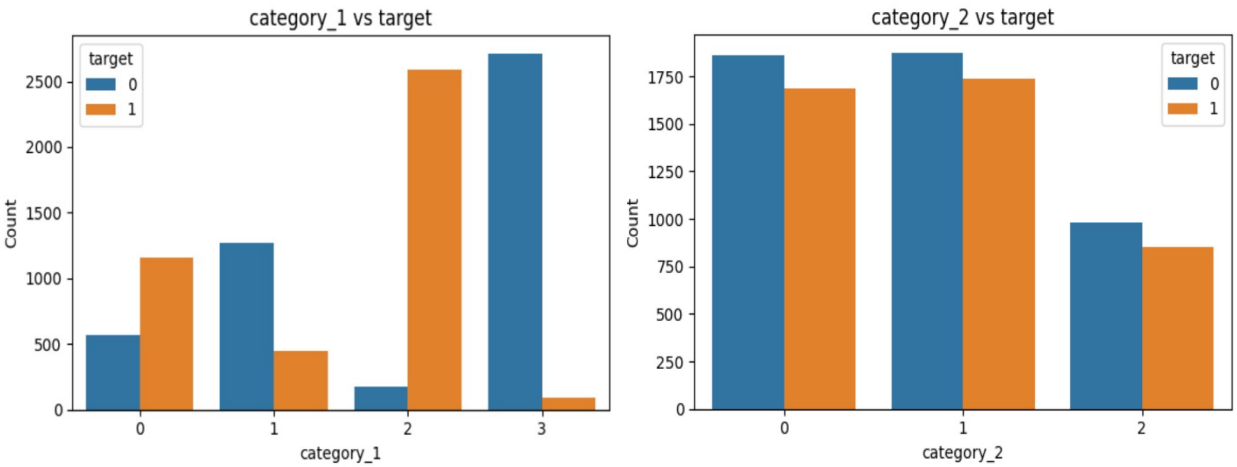






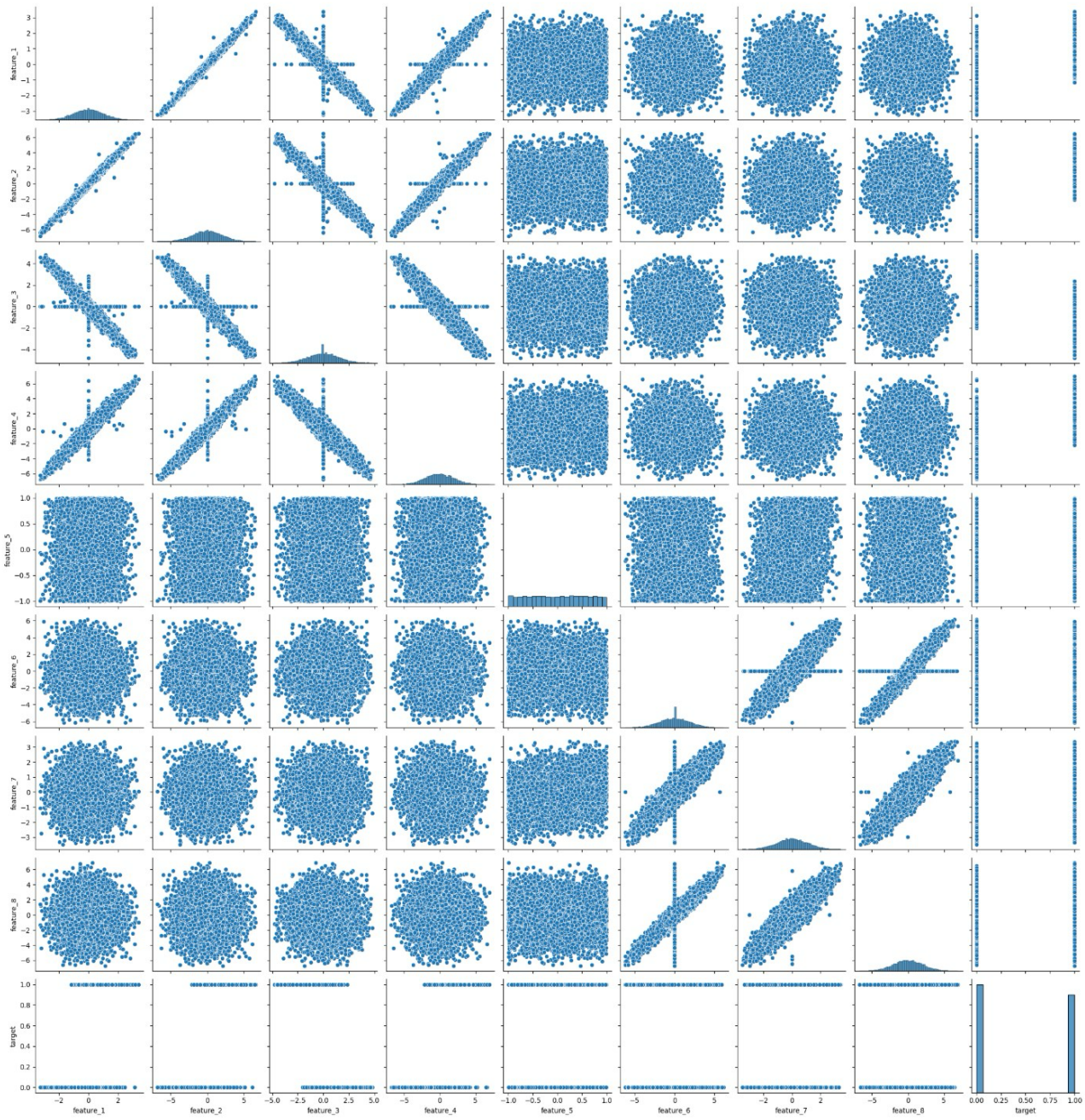
3.2) Analysis of Categorical Features

Categorical features were analyzed using bar graphs to visualize the frequency of each category. The bar plot indicates that a significant number of customers involved in specific category.



3.3) Pairwise Scatter Plot

Also we have visualized the relation with the help of pairwise scatter plot. Where we can identify the relational trends, actual trends in the data, and distribution and outliers. Here, we have mentioned the pairwise scatter plot below.

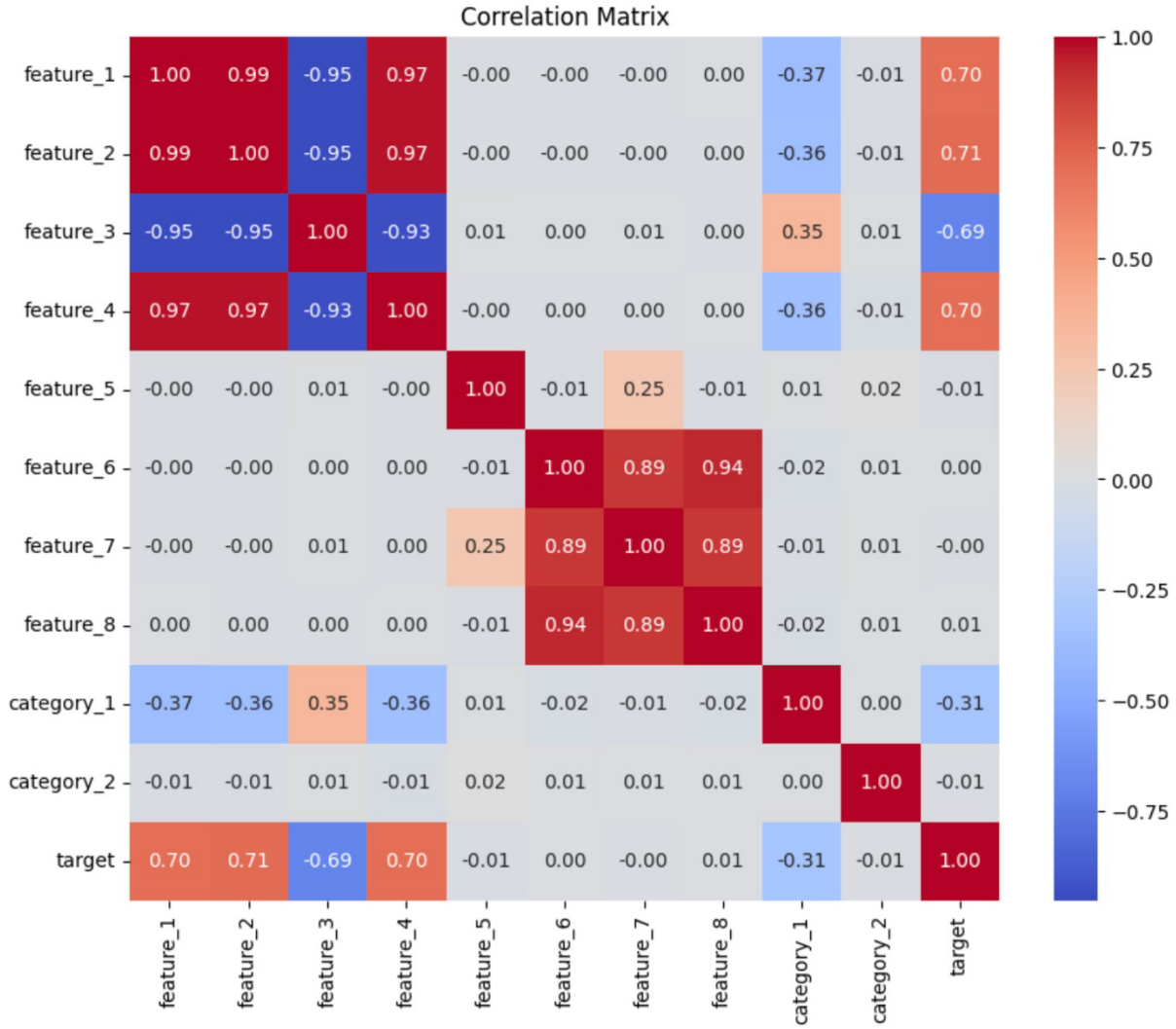


The pairplot shows how features are related to each other and to the target. Some features, like feature_1, feature_2, and feature_4, show clear linear relationships. Most other features are scattered randomly, showing weak or no strong relationship. Overall, a few feature pairs are correlated, while most are independent.

3.4) Correlation Analysis

The heatmap reveals that feature_1, 2 and 4 are almost perfectly inter-correlated and each shows a strong positive relationship with target while feature_3 is equally strongly but negatively correlated with those three. Features_6, 7 and 8 form their own tightly inter-correlated group but all three hover near zero correlation with target.

By correlation matrix, we can also look the percentage of dependencies of the features on other features. Because correlation coefficient ranges between -1 and 1. Where 1 mean strongly positive correlation and -1 means strongly negative correlation.



3.5) Hypothesis Testing

- **T-test:**

We compared the means of each numerical feature across the two classes. Features 1–4 all yielded extremely large |t| values (around 90) with p-values effectively zero, demonstrating that their distributions differ dramatically between the groups. Features 5–8, by contrast, produced t-statistics near zero and pvalues well above 0.05, indicating no significant mean differences.

Feature	t_statistic	p_value
feature_1	-93.7834	0.0000
feature_2	-94.7038	0.0000
feature_3	90.5290	0.0000
feature_4	-92.1744	0.0000
feature_5	0.7471	0.4550
feature_6	-0.1600	0.8729
feature_7	0.1163	0.9074
feature_8	-0.6072	0.5437

- **Chi-square test:**

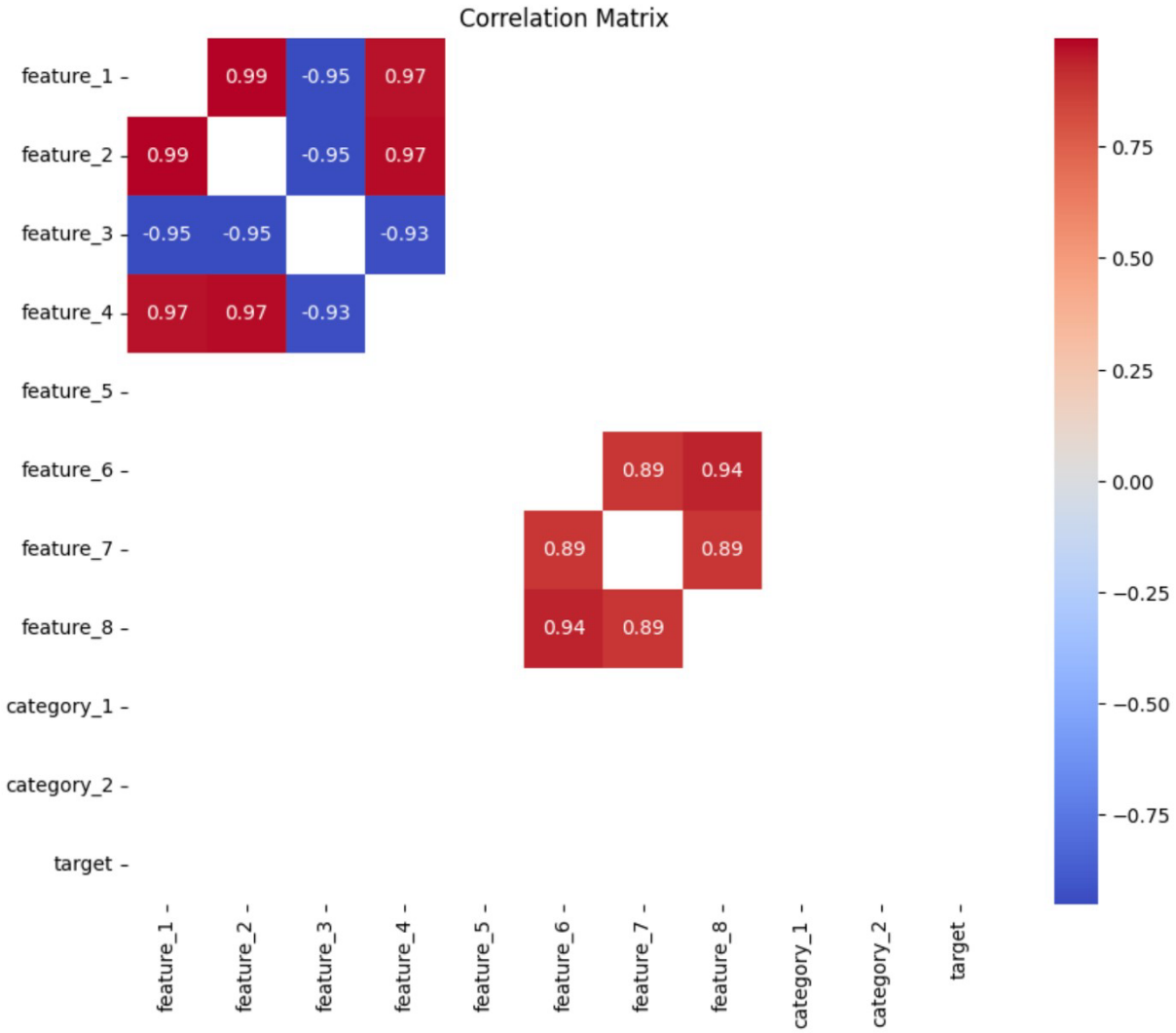
We tested for independence between each categorical feature and the label. For category_1, the chi2 statistic is 5175 ($p < 0.001$), showing a very strong association with the target classes. For category_2, chi2 ≈ 1.32 with $p \approx 0.52$, indicating no evidence of dependence.

Category	chi2_statistic	p_value
category_1	5175.3203	0.000
category_2	1.3193	0.517

4. Feature Engineering

Feature engineering involves creating new features from existing ones to improve model performance. This process is guided by domain knowledge and the insights gained from EDA.

To reduce redundancy among groups of highly correlated variables and amplify their joint signal, we collapsed each cluster into a single “meta-feature.” This both simplifies the model and preserves the bulk of the predictive power.



4.1) Combine Existing Features :

Cluster 1_2_3_4:

We observed that feature_1, feature_2, and feature_4 move almost lock-step (0.97), while feature_3 is equally strongly but negatively correlated (ρ ≈ –0.95). To align their directions, we first inverted feature_3 (feature_3_inv = –feature_3), then took the row wise average of the four.

Cluster 6_7_8:

Similarly, feature_6, feature_7, and feature_8 form a tight positive cluster (ρ ≈ 0.90). We averaged them directly.

4.2) Removal of Weak Features:

After evaluating the correlation between the newly engineered features and the target variable, the following features were removed due to their weak contribution:

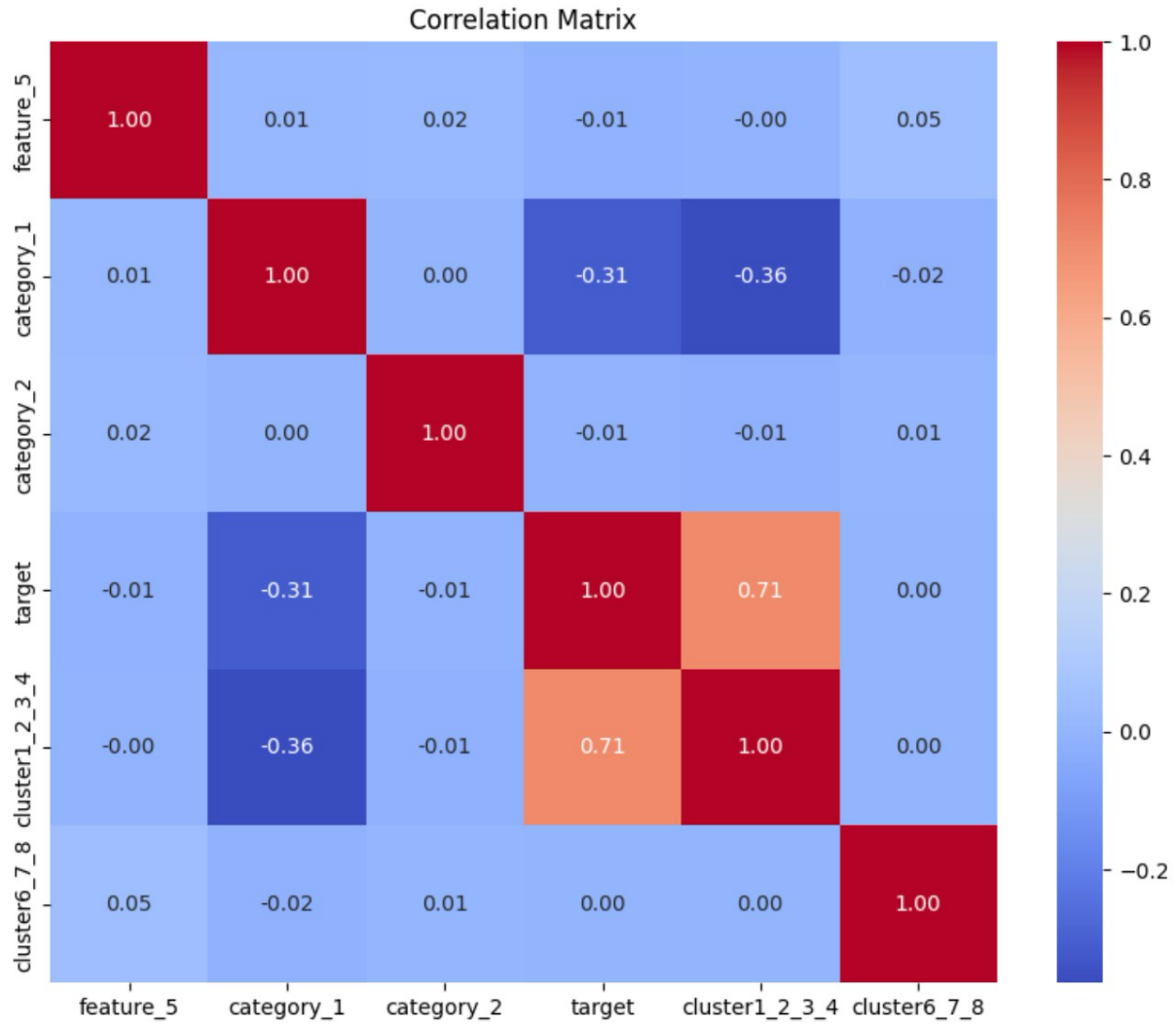
Feature_1, Feature_2, Feature_3, Feature_4, Feature_6, Feature_7, Feature_8, Feature_3_inv.

```
# Feature 3 is negatively correlated with the others.

data["feature_3_inv"]   = -data["feature_3"]
data["cluster1_2_3_4"]   = data[["feature_1","feature_2","feature_4","feature_3_inv"]].mean(axis=1)

# Cluster 2: features 6, 7, and 8
data["cluster6_7_8"]    = data[["feature_6", "feature_7", "feature_8"]].mean(axis=1)

# dropped the originals
to_drop = ["feature_1","feature_2","feature_3","feature_4",
           "feature_6","feature_7","feature_8","feature_3_inv"]
data = data.drop(columns=to_drop)


print(data.head())
```

```
   feature_5  category_1  category_2  target  cluster1_2_3_4  cluster6_7_8
0   0.784920           0           2       1        0.781187     -1.917261
1   0.704674           1           0       0        0.049615     -1.926697
2  -0.250029           2           2       1        1.128799      1.550019
3   0.882201           2           1       1        2.322824      2.991862
4   0.610601           1           2       0       -0.437824      0.248574
```

4.3) Correlation Analysis:

After adding new features, we evaluated their correlation with the target variable to assess their potential impact on the model's performance. The correlation between the new features:



Finally, we have feature_5, category_1, category_2, cluster_1_2_3_4 and cluster_6_7_8 in our dataset.

5. Modelling and Evaluation

We began by separating our engineered dataset into predictors (all columns except target) and the binary label. 70–30 stratified split ensured the training and test sets preserved class balance (randomstate=42).

5.1) Model training:

We tested four models for this task:

- Random Forest Classifier
- Logistic Regression Classifier
- Gradient Boost Classifier
- Decision Tree Classifier

5.2) Evaluation:

```python
def evaluate_model(model, X_train, y_train, X_test, y_test):

    model.fit(X_train, y_train)
    # Predictions
    predictions = model.predict(X_test)

    print("Classification Report:")
    print(classification_report(y_test, predictions))

    # Evaluation
    conf_matrix = confusion_matrix(y_test, predictions)

    # Visualization of Confusion Matrix
    plt.figure(figsize=(6, 4))
    sns.heatmap(pd.DataFrame(conf_matrix), annot=True, cmap="YlGnBu", fmt='g')
    plt.title('Confusion matrix')
    plt.ylabel('Actual label')
    plt.xlabel('Predicted label')
    plt.show()

    return model
```

✓ 0.0s

Each model was evaluated using key performance metrics:

- **Accuracy**: Overall correctness of the model
- **Precision**: True positives out of predicted positives
- **Recall**: True positives out of actual positives
- **F1 Score**: Harmonic mean of precision and recall

```python
# Check overfitting
def check_overfitting(model, X_train, y_train, X_test, y_test):
    train_accuracy = model.score(X_train, y_train)
    test_accuracy = model.score(X_test, y_test)
    print(f"Training Accuracy: {train_accuracy:.4f}")
    print(f"Testing Accuracy: {test_accuracy:.4f}")
```

✓ 0.0s

Computes and prints both **training accuracy** and **testing accuracy**. A large gap indicates overfitting.

5.3) Results:

- Random Forest:

```
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.87      0.85      1407
           1       0.85      0.81      0.83      1293

    accuracy                           0.84      2700
   macro avg       0.84      0.84      0.84      2700
weighted avg       0.84      0.84      0.84      2700
```

- Decision Tree:

```
Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.81      0.80      1407
           1       0.79      0.77      0.78      1293

    accuracy                           0.79      2700
   macro avg       0.79      0.79      0.79      2700
weighted avg       0.79      0.79      0.79      2700
```

- Logistic Regression:

```
Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.87      0.87      1407
           1       0.86      0.85      0.86      1293

    accuracy                           0.86      2700
   macro avg       0.86      0.86      0.86      2700
weighted avg       0.86      0.86      0.86      2700
```

- Gradient Boosting:

```
Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.88      0.87      1407
           1       0.86      0.85      0.86      1293

    accuracy                           0.86      2700
   macro avg       0.86      0.86      0.86      2700
weighted avg       0.86      0.86      0.86      2700
```

5.4) Discussion:

The project revealed several key insights:

Random Forest: A substantial boost over a single tree: f1 ≈ 0.84, precision and recall balanced at roughly 0.84 for both classes.

Decision Tree: Lowest overall accuracy and f1 (0.79), reflecting over-simplicity and overfitting. Precision and recall both hover around 0.79 for each class, indicating no bias but limited expressive power.

Logistic Regression: It shows strong linear baseline with f1 ≈ 0.86.Precision/recall around 0.86/0.85–0.87, illustrating well-calibrated probability estimates even without non-linear interactions.

Gradient Boosting: Matches Logistic Regression at ~86% accuracy and f1. higher recall for the positive class, suggesting slightly better sensitivity to class-1 examples. ensemble methods like Random Forest and Gradient Boosting outperform a solitary decision tree by a wide margin; logistic regression remains a very competitive, lowvariance alternative; and gradient boosting offers the best trade-off between precision and recall, particularly for the minority class.

6. Hyperparameter Tuning:

Tuning Methodology:

Optimize model hyperparameters to improve performance compared to default settings.

1. **GridSearchCV**: Used 5-fold cross-validation to test hyperparameter combinations.
2. **Models Tuned**: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting
3. **Evaluation Metrics**: Accuracy, Precision, Recall, F1-score, support.

The tuning process involved:

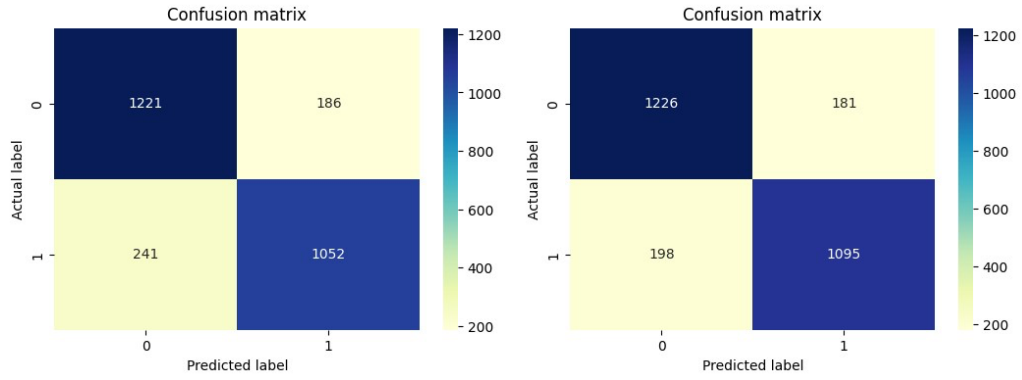
- Defining parameter grids specific to each model.
- Using GridSearchCV to search for the optimal combination of hyperparameters.
- Comparing performance between default and tuned models.

Visualizations:

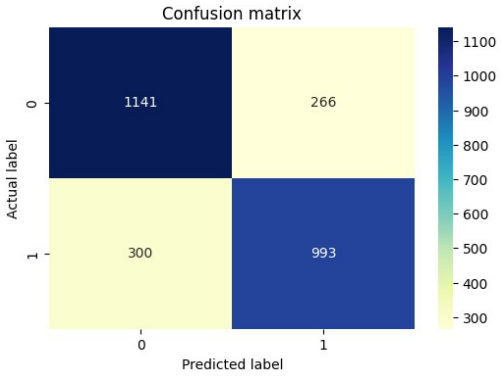
Confusion metrics for each model performance – normal and tunned:

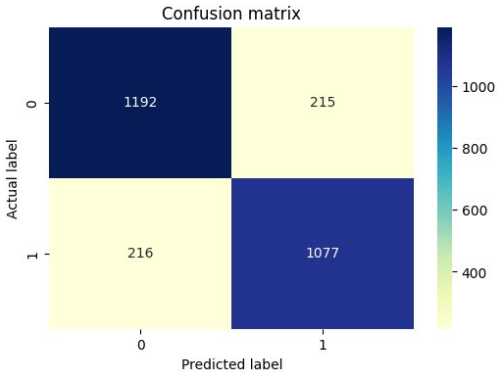
Default Tuned

- Random Forest:

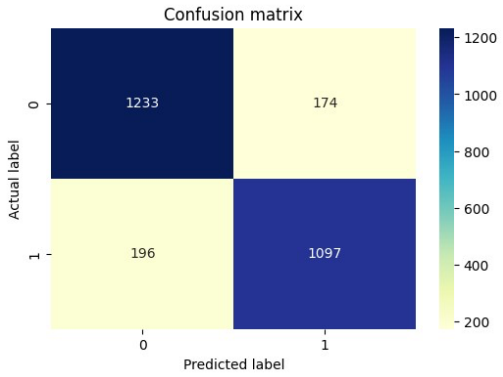


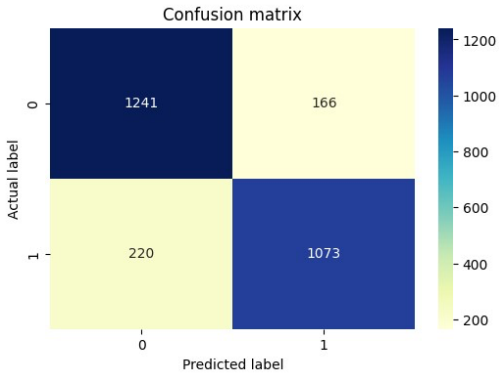
- Decision Tree:



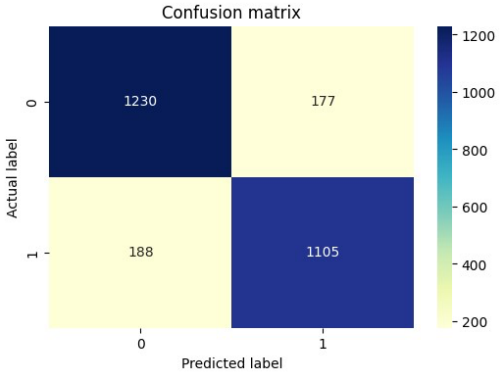


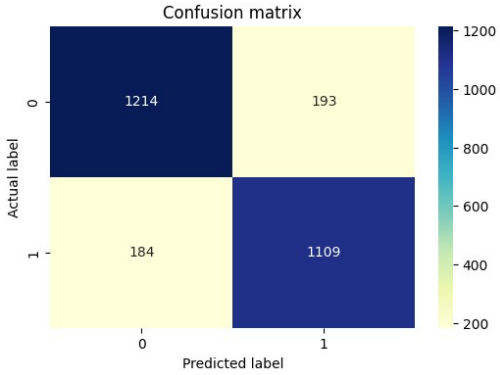
- Logistic Regression:





- Gradient Boosting:





Results:

Model	Normal	Tuned	Best Hyperparameters
Logistic Regression	85.00%	86.00%	C=1, penalty='l2', solver='liblinear'
Decision Tree	79.00%	86.00%	max_depth=None, min_samples_split=5, min_samples_leaf=2
Random Forest	84.00%	88.00%	n_estimators=200, min_samples_split=5, min_samples_leaf=2
Gradient Boosting	84.00%	88.00%	n_estimators=100, learning_rate=0.1, max_depth=3

- Tuning boosted all models by ~1–4 points; Random Forest and Gradient Boosting now share the top accuracy of 88.0%

7. Model Interpretation:

We used feature importance scores from the four models to understand which features were most predictive.

```
def feature_importance(model, selected_features):
    # Feature importances from Best Random Forest
    importances = model.feature_importances_
    feature_names = X[selected_features].columns
    feature_importance_df = pd.DataFrame({'feature': feature_names, 'importance': importances})
    print(feature_importance_df.sort_values(by='importance', ascending=False))
```

High-Importance Features:

- feature_2 was important in all models, especially Logistic Regression.
- Combined feature cluster1_2_3_4 was especially strong in ensemble models.

Moderate-Importance Features:

- feature_1 and feature_4 showed moderate influence depending on the model.

Low-Importance Features:

- feature_6, feature_7, and feature_8 had very low importance.
- Categorical features (category_1 and category_2) added little predictive power.

Trends by Model:

- **Logistic Regression**: Strong focus on linear features like feature_2 and derived averages.
- **Decision Tree / Random Forest**: Importance spread across multiple numeric features.
- **Gradient Boosting**: Balanced but like Logistic Regression in importance ranking.